Constitutional Synchronization

Executable Specification Through Governed Correspondence Among Meaning, Model, Realization, Evidence, Authority, and Projection

A system remains itself when every path from meaning to effect can still be made to answer for the others.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Software systems rarely possess one authoritative representation of themselves. They exist simultaneously as requirements, semantic models, schemas, transition rules, source code, binaries, runtime processes, tests, receipts, authority records, documentation, and user interfaces. These representations may begin in agreement and later diverge independently. An application can execute successfully while running unadmitted bytes, preserving obsolete semantics, relying upon stale evidence, exercising revoked authority, or presenting users with a projection that no longer corresponds to constitutional state.

This paper introduces **constitutional synchronization**, a domain-neutral framework for maintaining governed correspondence among the identity, meaning, lawful transitions, executable realizations, observed behavior, evidence, authority, history, dependencies, and public projections of a system. Constitutional synchronization does not collapse specification into implementation. It preserves their distinction while making the path between them explicit, identity-bearing, witnessed, and enforceable.

A system is constitutionally synchronized for a declared purpose when its current realization can be shown to correspond to its admitted model and transition law; its execution is adequately witnessed; its evidence supports its current standing; its authority remains valid; its dependencies and historical lineage are current; and its external projections faithfully expose the relevant constitutional state. The framework distinguishes synchronization from equality, divergence from insufficient evidence, local synchronization from configuration-level synchronization, and resynchronization from ordinary mutation.

The paper develops a formal model, a taxonomy of desynchronization, a system of constitutional courts, a synchronization-state vocabulary, and a governed

resynchronization process. It further examines executable specification, content-addressed intake, user-interface projection, recursive self-hosting, and constitutionally bounded self-improvement. The central claim is that the traditional gulf between specification and implementation is not overcome by eliminating either side, but by maintaining enforceable correspondence across every identity-bearing layer through which meaning becomes effect.

---

1. Introduction

A contemporary software system exists in many places at once.

It exists in the language used to describe its purpose. It exists in architecture diagrams, schemas, interface contracts, source repositories, compiled artifacts, deployment manifests, running processes, telemetry, tests, authorization records, documentation, and user interfaces. Each representation is partial. Each may change at a different time. Each may be maintained by a different person or machine. Each may claim to describe the same system.

They often do not.

A documentation page may describe an atomic transaction while the implementation performs several separately committed effects. A user interface may display a component as verified even though the verification applied only to a predecessor. A source hash may remain valid while the semantic regime under which the source was interpreted has changed. A package may be byte-identical to an admitted package but execute under a revoked authority. A test may continue to pass while observing the wrong process. A runtime may implement a useful capability that has never been incorporated into the governing model.

Traditional software engineering recognizes portions of this problem. Axiomatic reasoning relates programs to preconditions and postconditions. Design by contract relates components to assumptions and guarantees. Formal specification relates implementations to abstract behaviors. Runtime verification compares executions with temporal properties. Model-driven engineering and models-at-runtime preserve relationships between operational systems and their models. Modular and interface theories constrain composition. System-safety methods examine emergent hazards and inadequate control at the level of the complete system.

Yet these approaches commonly govern different correspondence surfaces. They do not ordinarily provide one vocabulary for asking whether meaning, identity, realization, observation, evidence, authority, history, and projection remain mutually accountable.

This paper proposes such a vocabulary.

Constitutional synchronization is the maintained, governed correspondence among the representations through which a system acquires identity, mean-

ing, executable form, evidence, permission, historical continuity, and external visibility.

The word **constitutional** is used technically. It refers to the highest-order rules under which a system determines:

- what its objects are;
- which transitions are lawful;
- what evidence is required;
- who or what may authorize reliance;
- which effects may become reachable;
- how succession is established;
- and how invalid or stale states are refused, isolated, demoted, or repaired.

The framework applies to software programs, applications, workflows, data systems, agent assemblies, scientific instruments, organizational protocols, and recursively self-hosting operating environments.

Its central thesis is:

> **Executable specification is not achieved by making specification and implementation identical. It is achieved by making every transition between meaning and effect explicit, identity-bearing, witnessed, and constitutionally enforceable.**

---

2. The Many Bodies of a System

The traditional specification–implementation distinction is too narrow for modern systems.

A system may possess at least the following bodies:

1. **Identity body** – the exact artifacts, objects, and histories that constitute the system.
2. **Semantic body** – what the system's concepts, interfaces, states, and outcomes mean.
3. **Normative body** – which invariants and transitions are permitted.
4. **Realization body** – the code, binaries, services, models, agents, and infrastructure that implement the system.
5. **Operational body** – what the realization actually does in a particular environment.
6. **Evidentiary body** – the tests, traces, receipts, proofs, observations, and replications supporting claims about the system.
7. **Authority body** – the standing, jurisdiction, delegation, admission, and effect permissions under which the system operates.
8. **Historical body** – the predecessor relationships, amendments, migrations, revocations, and prior states that explain how the system became current.

9. **Dependency body** – the actual components, environmental guarantees, toolchains, and mediated relations upon which behavior depends.
10. **Projection body** – what users, agents, interfaces, APIs, documentation, and dashboards are shown.

These bodies are neither identical nor interchangeable.

A binary hash identifies bytes. It does not identify their meaning, authority, or lawful role. A passing test establishes only what the test is capable of observing. A current specification does not prove that the deployed runtime realizes it. A user interface can correctly render stale state. A valid local component can participate in an invalid global configuration.

A system therefore cannot be synchronized merely by checking that source code matches a repository or that an implementation passes tests. Synchronization is a multidimensional correspondence claim.

---

3. Definition

3.1 Canonical definition

> **Constitutional synchronization is the condition in which the identity, meaning, lawful transitions, executable realization, observed behavior, evidence, standing, authority, history, dependencies, and projections of a system remain adequately corresponding for a declared purpose throughout a declared disturbance envelope.**

The term **adequately corresponding** is intentional. Synchronization does not require byte equality or representational identity among layers. A semantic model, a binary, an execution trace, and a user-interface card are different kinds of objects. They cannot be equal.

They can, however, remain accountable to one another through explicit correspondence relations.

3.2 Synchronization is purpose-relative

A system is not synchronized in the abstract. It is synchronized for some purpose (p), some invariant set (K), some context (Γ), and some disturbance envelope (Λ).

A component may be synchronized for:

- read-only historical inspection;
- experimental execution;
- local development;
- noncritical production work;

- safety-critical control;
- public evidentiary reliance.

The evidence sufficient for one purpose may be inadequate for another.

Thus:

$$\text{Synchronized}(S)$$

is generally incomplete. The fuller form is:

$$\text{Synchronized}_{\Gamma}(S, p, K, \Lambda)$$

3.3 Synchronization is not equality

Let a system state be represented as:

$$\mathcal{S} = \langle I, M, T, R, O, E, A, H, D, P, \Gamma, \Lambda \rangle$$

where:

Symbol	Meaning
(I)	Identity and constitutive objects
(M)	Meaning, model, and declared purpose
(T)	Lawful transition structure
(R)	Executable realization
(O)	Observed behavior
(E)	Evidence and witness
(A)	Standing, authority, admission, and jurisdiction
(H)	History and lineage
(D)	Dependencies and environmental guarantees
(P)	Human- and machine-facing projections
(Γ)	Current context
(Λ)	Disturbance envelope

Constitutional synchronization does not assert:

$$I = M = T = R = O = E = A = H = D = P$$

It asserts that relevant correspondence obligations among them are satisfied.

A simplified synchronization condition is:

$$\begin{aligned}\text{Synchronized}_{\Gamma}(\mathcal{S}, p, K, \Lambda) \iff & \text{IdentityBound}(I, R, H) \\ & \wedge \text{SemanticallyCorresponds}(M, R, p) \\ & \wedge \text{TransitionConforms}(T, R, H) \\ & \wedge \text{ObservedUnder}(O, R, \Gamma, \Lambda) \\ & \wedge \text{EvidenceAdequate}(E, O, p) \\ & \wedge \text{AuthorityCurrent}(A, R, p) \\ & \wedge \text{DependenciesAccounted}(D, R, \Gamma) \\ & \wedge \text{ProjectionFaithful}(P, I, M, O, E, A, H) \\ & \wedge \text{ConfigurationAdmissible}(\mathcal{S}, p)\end{aligned}$$

Each conjunct is independently falsifiable. No single result substitutes for all of them.

---

4. Constitutional Objects and Synchronization Edges

4.1 Synchronization as a graph

A constitutionally modeled system may be represented as a graph:

$$G_{\text{sync}} = (V_{\text{sync}}, E_{\text{sync}})$$

The vertices are identity-bearing constitutional objects:

- specifications;
- schemas;
- source artifacts;
- binaries;
- compiler profiles;
- runtime instances;
- observations;
- receipts;
- authority records;
- standing records;
- historical states;
- interface projections.

The edges are typed correspondence obligations:

- `IDENTIFIES`;
- `REALIZES`;
- `IMPLEMENTS`;
- `OBSERVES`;
- `SUPPORTS`;

- AUTHORIZES;
- DEPENDS_ON;
- SUPERSEDES;
- PROJECTS;
- TRANSLATES;
- CONSTRAINS;
- REVOKES.

A system becomes difficult to govern when these edges exist only informally.

For example:

```
specification.md
    REALIZED_BY
runtime.bin

runtime.bin
    PRODUCED_BY
compiler-profile-17

receipt-884
    OBSERVES
runtime.bin

admission-29
    AUTHORIZES
runtime.bin

dashboard-card-12
    PROJECTS
admission-29
```

The graph makes questions of drift precise.

If `runtime.bin` changes, which receipts become stale? If the compiler profile is revoked, which realizations lose standing? If an admission expires, which interface controls must become unreachable? If a projection still points to a superseded receipt, which users are receiving a false constitutional view?

4.2 Synchronization vectors

Because synchronization is multidimensional, the system may maintain a synchronization vector:

$$\begin{aligned}\sigma(\mathcal{S}, p) = \langle &\sigma_I, \sigma_M, \sigma_T, \sigma_R, \sigma_O, \\ &\sigma_E, \sigma_A, \sigma_H, \sigma_D, \sigma_P, \\ &\sigma_C \rangle\end{aligned}$$

where (σ_C) represents configuration-level synchronization.

Each dimension may hold a typed status rather than a Boolean:

- demonstrated;
- demonstrated with limits;
- stale;
- divergent;
- contradicted;
- unproven;
- unavailable;
- historical only;
- outside jurisdiction.

This prevents a convenient aggregate status from concealing which correspondence has failed.

4.3 Identity is necessary but insufficient

Cryptographic commitment can establish that an artifact has not changed relative to a recorded digest.

It cannot establish:

- that the artifact means what a model claims;
- that it was produced through the declared toolchain;
- that it remains authorized;
- that its evidence is current;
- that its effects are permissible;
- that it remains suitable for the present context.

Thus:

$$\text{DigestMatch} \not\Rightarrow \text{ConstitutionalSynchronization}$$

Content identity supplies a stable subject for synchronization claims. It does not settle those claims.

---

5. Constitutional Synchronization and Executable Specification

5.1 The traditional gulf

The conventional specification–implementation gap is often described as the distance between what a system is intended to do and what its code actually does.

Formal methods narrow this distance by representing intended behavior mathematically and proving or checking relationships between abstract models and implementations. Hoare logic, temporal logic, refinement, contracts, interface theories, and executable modeling environments each provide important machinery for portions of that problem.

Constitutional synchronization expands the problem.

A system may correctly implement a functional specification while remaining constitutionally desynchronized because:

- the implementation is not the admitted implementation;
- its authority has expired;
- its dependencies differ from those analyzed;
- its observation record is stale;
- its public interface overstates its standing;
- its semantic context has changed;
- its transition was unauthorized;
- its configuration produces higher-order effects absent from the local specification.

5.2 Governed closure rather than collapse

Specification and implementation should remain distinct.

If they became indistinguishable, correspondence could no longer be tested.

The desired relationship is:

$$\text{Specification} \neq \text{Implementation}$$

but:

$$\text{Specification} \xrightarrow[\text{witnessed}]{\text{authorized realization}} \text{Implementation}$$

The traditional gulf is not eliminated by merging the shores. It is closed by a governed bridge.

That bridge contains:

- exact identities;
- declared semantics;
- lawful transition records;
- realization commitments;
- execution witnesses;
- evidence;
- standing;
- authority;
- rollback;
- historical succession.

5.3 Reachability gives specification force

A specification becomes executable in the constitutional sense when it governs which realizations may become operationally reachable.

The critical invariant is:

$$\neq g\,\text{Admitted}(R, p)$$
$$\Rightarrow$$
$$\neq g\,\text{ReachableAsConstitutionalBehavior}(R, p)$$

Candidate code may exist. It may compile. It may run inside a sandbox. It may produce useful results.

It does not thereby become the active system.

This enforced asymmetry between candidate bytes and constitutional reality gives specification practical force. The model is not merely descriptive documentation. It constrains admission and reachability.

5.4 Semantic gravity

This paper calls the resulting effect **semantic gravity**.

Semantic gravity is the architectural tendency for artifacts, transitions, and effects to remain attached to the identities, meanings, evidence, and authority that justify them.

It arises from the combined operation of:

- content-addressed identity;
- fail-closed intake;
- explicit transition models;
- realization correspondence;
- witnessed execution;
- standing and authority gates;
- historical lineage;
- projection discipline.

Without semantic gravity, artifacts drift toward operational use because they are available, convenient, or apparently functional.

With semantic gravity, operational adoption requires lawful placement.

---

6. Constitutional Desynchronization

6.1 Desynchronization is a family of states

The opposite of synchronization is not one undifferentiated failure.

Constitutional desynchronization is any condition in which one or more required correspondence relations are divergent, stale, unproven, contradictory, or outside their declared context.

A program may continue to execute while desynchronized. Indeed, some of the most dangerous desynchronizations preserve apparent functionality.

6.2 Identity desynchronization

Identity desynchronization occurs when the system cannot establish that the operational artifact is the artifact to which its evidence, standing, or authority applies.

Examples include:

- path substitution;
- mutable dependencies;
- stale generated files;
- an installed binary differing from the admitted binary;
- a receipt pointing to a predecessor;
- a package name reused for different bytes.

Typical disposition:

`IDENTITY_DIVERGENCE`

Identity divergence should ordinarily fail closed because every downstream claim has lost its stable subject.

6.3 Semantic desynchronization

Semantic desynchronization occurs when labels, interfaces, or state names remain stable while their meaning changes.

Examples include:

- `active` changing from "installed and admitted" to "currently executing";
- a timestamp changing from event time to ingestion time;
- `authorized` being used for both delegated authority and mere technical reachability;
- a field retaining its name after changing units or population scope;
- the same concept being merged across incompatible disciplinary regimes.

Typical disposition:

`SEMANTIC_DIVERGENCE`

Semantic drift is especially dangerous because syntactic composition may continue to succeed.

6.4 Specification–realization desynchronization

This occurs when the executable realization no longer corresponds to the admitted model or transition law.

Examples include:

- an API accepting inputs beyond its contract;
- a supposedly atomic operation committing partial effects;
- failure semantics changing during refactoring;
- an optimization removing an invariant-preserving check;
- runtime behavior added without model amendment.

Two directional states are useful:

- `SPECIFICATION_AHEAD_OF_REALIZATION`;
- `REALIZATION_AHEAD_OF_SPECIFICATION`.

The second may describe a useful implementation innovation, but usefulness does not grant authority.

6.5 Realization–observation desynchronization

This occurs when the observation system does not adequately capture the behavior it claims to witness.

Examples include:

- monitoring the wrong process;
- a success receipt emitted before commit;
- effects occurring outside the instrumented boundary;
- logs omitting rejected or rolled-back operations;
- a test observing a mock while production uses another pathway.

Typical dispositions include:

- `WITNESS_INSUFFICIENT`;
- `OBSERVATION_DIVERGENCE`.

Runtime verification provides important foundations for observing executions against declared properties, but constitutional synchronization additionally binds observations to identity, evidence, standing, authority, and projection.

6.6 Evidence desynchronization

Evidence desynchronization occurs when a claim continues to be presented as standing-bearing after its evidence has become stale, inapplicable, derivative, contradicted, or invalidated.

Examples include:

- tests applying only to an older revision;
- an external source being corrected or withdrawn;
- apparently independent sources being traced to one origin;
- a disturbance envelope expanding beyond what was tested;
- a proof applying to a model no longer realized.

Typical states include:

- `EVIDENCE_STALE`;
- `STANDING_EXCEEDS_EVIDENCE`;
- `SYNCHRONIZATION_UNPROVEN`.

Unproven synchronization is not the same as demonstrated divergence. The distinction is epistemically necessary.

6.7 Authority desynchronization

Authority desynchronization occurs when current executable reachability and current authority no longer agree.

Examples include:

- revoked code remaining active;
- expired delegation continuing to authorize effects;
- local experimental admission being treated as global production standing;
- a composition acquiring effective authority absent from its constituent grants;
- a user interface permitting an operation no longer admitted.

Typical disposition:

`AUTHORITY_DIVERGENCE`

An artifact may remain technically capable and evidentially supported while lacking current permission.

6.8 Temporal desynchronization

Temporal desynchronization occurs when objects from different historical states are treated as contemporaneous.

Examples include:

- current code with an obsolete schema;
- a revised dataset interpreted through the original publication notes;
- a new authority state paired with a prior dependency graph;
- current UI labels backed by historical court results;
- a package update combined with a predecessor's admission receipt.

Typical dispositions include:

- `TEMPORAL_MISALIGNMENT`;
- `HISTORICAL_ONLY`;
- `RESYNCHRONIZATION_REQUIRED`.

6.9 Dependency desynchronization

Dependency desynchronization occurs when declared dependency topology differs from actual dependency topology.

Examples include:

- hidden runtime imports;
- undocumented environmental guarantees;
- a new network dependency;
- a supposedly optional service becoming operationally mandatory;
- a compiler or interpreter identity changing beneath a stable build script;
- multiple independent witnesses relying upon one hidden common implementation.

Typical disposition:

`DEPENDENCY_DRIFT`

Parnas's treatment of modular decomposition and later work on architectural mismatch both show why hidden design decisions and incompatible assumptions undermine apparently composable systems. Constitutional synchronization makes those assumptions identity-bearing synchronization obligations.

6.10 Projection desynchronization

Projection desynchronization occurs when a user interface, API, report, documentation surface, or agent context presents a state different from current constitutional reality.

Examples include:

- showing `VERIFIED` when the object is only a candidate;
- displaying a historical result as current;
- leaving an action enabled after authority revocation;
- documenting an unadmitted capability;
- suppressing residuals that materially limit a claim;
- representing an unavailable court as passing.

Typical disposition:

`PROJECTION_STALE`

Projection drift is not merely a cosmetic documentation defect. It alters the information upon which users and agents make decisions.

6.11 Configuration desynchronization

Configuration desynchronization occurs when components are locally synchronized but their composition is not.

Examples include:

- circular dependency;
- semantic contradiction across interfaces;
- authority amplification;
- correlated failure;

- unbounded resource demand;
- information leakage produced only by combination;
- two current components depending upon incompatible historical regimes.

This yields the non-implication:

$$(\forall C_i \in \Sigma : \text{Synchronized}(C_i)) \not\Rightarrow \text{Synchronized}(\Sigma)$$

Constraint-Bearing Composability separately establishes why locally valid and compatible components may still form a globally inadmissible configuration. Constitutional synchronization extends that principle by asking whether the complete configuration's meanings, histories, evidence, authority, and projections remain mutually aligned.

---

7. Synchronization State Vocabulary

A practical system should not compress synchronization into `true` or `false`.

A useful state vocabulary includes:

- **State:** `SYNCHRONIZED`
  Meaning: All required correspondence obligations are currently demonstrated
- **State:** `SYNCHRONIZED_WITH_DECLARED_LIMITS`
  Meaning: Correspondence holds only within explicit constraints
- **State:** `SYNCHRONIZATION_UNPROVEN`
  Meaning: No divergence is established, but evidence is inadequate
- **State:** `WITNESS_STALE`
  Meaning: Current realization lacks current observation
- **State:** `SPECIFICATION_AHEAD_OF_REALIZATION`
  Meaning: The admitted model has not yet been fully realized
- **State:** `REALIZATION_AHEAD_OF_SPECIFICATION`
  Meaning: Executable behavior exists without admitted model coverage
- **State:** `IDENTITY_DIVERGENCE`
  Meaning: Operational identity differs from committed identity
- **State:** `SEMANTIC_DIVERGENCE`
  Meaning: Meaning differs across supposedly shared representations
- **State:** `AUTHORITY_DIVERGENCE`
  Meaning: Reachability and authority disagree
- **State:** `DEPENDENCY_DRIFT`
  Meaning: Actual dependencies differ from declared dependencies

- **State:** `PROJECTION_STALE`
  Meaning: External representation differs from constitutional state
- **State:** `TEMPORAL_MISALIGNMENT`
  Meaning: Objects from incompatible historical states are combined
- **State:** `CONFIGURATION_SPLIT_BRAIN`
  Meaning: Locally current components form a globally incoherent assembly
- **State:** `HISTORICAL_ONLY`
  Meaning: State remains valid only as historical evidence
- **State:** `QUARANTINED_PENDING_RESYNCHRONIZATION`
  Meaning: Operational reachability is restricted during repair
- **State:** `UNRESOLVABLE_UNDER_CURRENT_MODEL`
  Meaning: Existing ontology cannot express or repair the divergence

These states imply different remedies.

`PROJECTION_STALE` may require projection regeneration. `WITNESS_STALE` may require renewed observation. `IDENTITY_DIVERGENCE` may require immediate isolation. `UNRESOLVABLE_UNDER_CURRENT_MODEL` requires modeling work rather than an implementation patch.

---

8. Constitutional Synchronization Courts

8.1 Courts as correspondence mechanisms

A constitutional court is an identity-bearing mechanism that evaluates a bounded proposition under declared evidence, context, and authority.

A synchronization court does not ask merely whether "the system works." It asks which correspondence relation has been established.

8.2 Court families

Identity Court Question:

> Is the object being evaluated the exact object to which the claim applies?

Evidence may include:

- content hashes;
- package commitments;
- compiler and toolchain identities;
- manifest membership;
- historical lineage.

Semantic Court Question:

> Do the participating models, labels, interfaces, and data objects carry compatible meanings for the claimed use?

Evidence may include:

- typed semantic models;
- units;
- taxonomies;
- mapping contracts;
- translation-loss records;
- contextual regimes.

Transition Court Question:

> Was the movement from predecessor to successor permitted by the applicable transition law?

Evidence may include:

- predecessor identity;
- proposal;
- authority;
- transition contract;
- mutation envelope;
- successor commitment.

Realization Court Question:

> Does the executable artifact realize the admitted model and operation contract?

Evidence may include:

- source and binary identities;
- compiler profile;
- invocation;
- expected postconditions;
- actual outputs;
- bounded execution trace.

Witness Court Question:

> Did the observation mechanism adequately observe the relevant realization and effects?

Evidence may include:

- trace coverage;
- process identity;

- environment;
- timing;
- instrumentation boundaries;
- negative-control cases.

Evidence and Standing Court Question:

> Does the available evidence justify relying upon the object as a bounded component for the declared purpose?

Evidence may include:

- independent replication;
- mutation results;
- residual assumptions;
- contradiction status;
- disturbance-envelope coverage.

Authority Court Question:

> Is the claimed operation currently admitted within the correct jurisdiction and effect class?

Evidence may include:

- delegation lineage;
- current authority state;
- revocation status;
- jurisdiction;
- purpose;
- effect authorization.

Projection Court Question:

> Does the human- or machine-facing projection faithfully represent current constitutional state?

Evidence may include:

- source state identity;
- projection policy;
- displayed standing;
- current court result;
- omission rules;
- stale-state behavior.

Configuration Court Question:

> Does the assembled system remain synchronized and admissible at configuration scale?

Evidence may include:

- dependency graph;
- semantic graph;
- aggregate authority;
- information-flow paths;
- resource ceilings;
- correlated witness dependencies;
- historical compatibility.

8.3 A court census

A constitutional system should maintain a census describing each court's meaning rather than exposing raw test filenames.

A court record should include:

- court identity;
- proposition established;
- explicit nonclaims;
- jurisdiction;
- affected subsystem;
- applicable phase;
- current standing;
- latest result;
- current synchronization status;
- evidence requirements;
- dependency courts;
- supported toolchains and environments;
- body-access requirements;
- historical predecessors;
- supersession status;
- links to receipts, source, failures, and related courts.

This provides a stable semantic layer for user interfaces, agents, schedulers, and audit tools.

8.4 Body-dependent and digest-sufficient courts

Some courts can operate from commitments and compact state.

Others require artifact bodies.

A **digest-sufficient** court may establish:

- exact identity;
- manifest membership;
- recorded standing;
- declared dependency;
- historical linkage.

A **body-dependent** court may require:

- source traversal;
- binary inspection;
- runtime trace;
- evidence contents;
- schema interpretation;
- semantic comparison.

The distinction prevents a digest from being treated as proof of properties that only body inspection can establish.

---

9. Resynchronization

9.1 Resynchronization is not ordinary repair

A patch changes an artifact.

A resynchronization changes the correspondence graph so that identity, meaning, realization, evidence, authority, history, and projection become mutually accountable again.

This may require changing:

- code;
- specification;
- standing;
- authority;
- dependencies;
- projections;
- or the ontology under which the discrepancy is expressed.

9.2 Resynchronization sequence

A governed resynchronization process may proceed as follows:

1. **Detect divergence.** A court, witness, user, or agent identifies a failed or stale synchronization edge.
2. **Bind the affected state.** The relevant predecessor identities, evidence, authority, and projections are frozen.
3. **Classify the divergence.** The system distinguishes identity, semantic, realization, evidence, authority, temporal, dependency, projection, and configuration defects.
4. **Constrain reachability.** Unsafe or misleading behavior is quarantined, narrowed, or marked historical.

5. **Generate exigence.** The divergence becomes a typed attention object with custody and dependency.
6. **Propose a target synchronized state.** The proposal declares which representations will change and which invariants must remain.
7. **Realize the successor.** Candidate code, models, evidence, or projections are produced below standing.
8. **Run synchronization courts.** Each affected correspondence is re-evaluated.
9. **Evaluate configuration consequences.** Local repair is not accepted if the complete assembly becomes inadmissible.
10. **Admit or refuse the successor.** The transition receives standing and authority only if required courts pass.
11. **Regenerate projections.** Human and machine views are derived from the admitted successor state.
12. **Preserve the divergence historically.** The prior state and repair remain inspectable.

9.3 Resynchronization can change the specification

The implementation is not always the defective side.

An implementation may reveal:

- an omitted requirement;
- an unmodeled environmental constraint;
- a useful capability absent from the specification;
- an invariant that cannot be realized;
- an ontology incapable of expressing the actual system.

The lawful response may therefore be to amend the model.

However:

$$\text{UsefulUnmodeledBehavior} \not\Rightarrow \text{AuthorizedSpecificationChange}$$

Model revision remains a separate constitutional transition.

---

10. Projection and Human Interfaces

10.1 Projection is part of the system

A user interface is often treated as an external presentation layer.

Constitutional synchronization treats it as an operational projection of system meaning.

A projection may affect:

- user expectations;
- available actions;
- perceived standing;
- trust;
- authority;
- attention;
- interpretation of failure.

It therefore participates in constitutional behavior.

10.2 Governed projection

A projection should be derived from:

$$P = \text{Project}(I, M, O, E, A, H, \Pi)$$

where (Π) is an explicit projection policy.

The projection policy may determine:

- which information is shown;
- which details are summarized;
- which historical states are available;
- how uncertainty is represented;
- what actions are enabled;
- how stale or divergent states are displayed.

The UI may simplify. It may not silently change standing or meaning.

10.3 Court census projection

A constitutionally synchronized court census would allow a UI to display:

- what each court proves;
- what it cannot prove;
- its latest result;
- whether that result is current;
- its dependencies;
- evidence and toolchain requirements;
- body-dependent or digest-sufficient status;
- historical and superseded results;
- affected subsystem and certification band;
- related failures and receipts.

The UI becomes a governed view of court meaning rather than an independent metadata system.

10.4 Human adequacy remains externally witnessed

Mechanical synchronization can establish that a UI faithfully projects constitutional state.

It cannot alone establish that the projection is:

- understandable;
- appropriately salient;
- cognitively manageable;
- correctly named for its audience;
- adequate for human decision-making.

Thus:

$$\text{ProjectionFaithful} \not\Rightarrow \text{HumanlyAdequate}$$

Human-interface adequacy requires human evidence.

---

11. Local and Configuration-Level Synchronization

11.1 The local fallacy

A system may contain components that are each individually:

- correctly identified;
- semantically coherent;
- well tested;
- currently admitted;
- faithfully projected.

Their combination may still be desynchronized.

Examples include:

- two services using different definitions of the same state;
- two individually authorized capabilities producing unauthorized aggregate power;
- a current component depending on a historically incompatible schema;
- two independent-looking witnesses sharing one hidden implementation;
- individually safe data releases enabling prohibited inference when joined.

11.2 Configuration synchronization

A configuration is constitutionally synchronized when:

- its components remain individually synchronized;
- its relations preserve declared semantics;
- aggregate authority remains justified;
- dependency topology is current;
- shared context is consistent;

- configuration-level invariants hold;
- projections do not conceal higher-order consequences.

This requires a separate configuration court.

Pairwise correspondence is insufficient because some properties arise only from:

- cycles;
- paths;
- accumulation;
- scale;
- shared dependency;
- mediation;
- historical interaction.

11.3 Synchronization and composability

Constraint-Bearing Composability asks what carries the invariants by which a composition remains valid and admissible. Constitutional synchronization asks whether the representations and judgments supporting that composition remain current and mutually accountable.

The frameworks are complementary.

A composition may have adequate constraint carriage at one time and later become desynchronized because:

- its evidence becomes stale;
- a mediator changes;
- authority expires;
- a projection is not updated;
- a dependency is replaced;
- a semantic regime shifts.

Likewise, synchronized components may form a configuration whose constraint carriage is inadequate.

---

12. Recursive Self-Hosting

12.1 Synchronizing the synchronizer

The most consequential application arises when a constitutional system represents and modifies itself.

A self-hosting system must synchronize:

- the specification of its admission machinery;
- the implementation of that machinery;
- the witness observing it;

- the authority permitting change;
- the historical transition;
- the projections used by operators.

The danger is self-ratification.

12.2 Role separation

A sound recursive process preserves:

$$\begin{aligned} &\text{Proposal} \\ &\neq \text{Investigation} \\ &\neq \text{Witness} \\ &\neq \text{Admission} \\ &\neq \text{Effect} \end{aligned}$$

These distinctions may be implemented through different humans, models, programs, or institutions.

Where one language model performs several roles, separation can still be meaningful if each invocation is governed by:

- role-specific context;
- explicit information exposure;
- blinded inputs where appropriate;
- distinct output contracts;
- restricted authority;
- immutable task commitments;
- independently checkable receipts.

The relevant property is not merely agent identity. It is informational and constitutional independence.

12.3 Bootstrap sequence

A constitutional bootstrap may proceed through:

1. externally hosted seed;
2. self-representation;
3. self-observation;
4. decomposition into bounded constitutional components;
5. candidate native realizations;
6. correspondence testing against the incumbent;
7. shadow operation;
8. bounded substitution;
9. constitutional cutover;
10. historical preservation of the seed.

The fixed point is reached when:

$$\mathrm{CGSOS} * n \to \mathrm{CGSOS} * n + 1$$

is itself a constitutionally synchronized transition.

12.4 Independent anchors

The system cannot prove its deepest courts solely by invoking those same courts.

Bootstrapping therefore requires partially independent anchors:

- a small seed verifier;
- alternate realizations;
- historical replay;
- negative mutation tests;
- frozen external commitments;
- detached witnesses;
- human or physical outcomes where appropriate.

The closer a component lies to identity, authority, admission, witness, or effect reachability, the stronger the required independence.

---

13. Constitutionally Bounded Self-Improvement

13.1 From synchronization failure to exigence

A desynchronization can become a structured work object:

$$\begin{aligned} &\text{divergence} \\ &\to \text{exigence} \\ &\to \text{investigation} \\ &\to \text{modeled correction} \\ &\to \text{realization} \\ &\to \text{witness} \\ &\to \text{admission} \end{aligned}$$

This creates a lawful self-improvement loop.

13.2 Positive feedback

An admitted improvement may improve the machinery that produces later improvements:

- better observation yields better diagnosis;
- better diagnosis yields better attention allocation;
- better models yield better realizations;

- better realizations reduce witness ambiguity;
- better evidence improves admission;
- better projections improve human governance.

This is genuine recursive acceleration.

13.3 Why "optimization" is incomplete

A constitutionally synchronized system should not maximize one scalar objective at the expense of standing, evidence, authority, reversibility, or human purpose.

It performs **governed evolution**.

A faster implementation that conceals residuals or weakens refusal may be constitutionally worse.

13.4 Three recursive loops

A useful distinction is:

Operational loop Changes scheduling, caching, resource allocation, retry policies, and other runtime behavior without changing admitted meaning.

Engineering loop Changes implementations while preserving admitted models and constitutional invariants.

Constitutional loop Changes models, capability boundaries, schemas, standing rules, authority structures, or amendment procedures.

Automation may increase from the constitutional loop toward the operational loop. Constitutional change should remain the most demanding.

---

14. Relationship to Prior Work

Constitutional synchronization does not replace formal specification, contracts, interface theories, runtime verification, models-at-runtime, modularity, or system-safety engineering.

It builds upon their central insights while broadening the correspondence problem.

Hoare's axiomatic basis relates program execution to logical assertions. Design by Contract makes assumptions and guarantees explicit. TLA+ and related temporal methods specify system behavior and refinement. Interface automata and contract-based design formalize compatibility and composition.

Modularity and architectural-mismatch research show why hidden design decisions and incompatible assumptions undermine composition. Runtime verification examines executions against declared properties. Models-at-runtime

research maintains causally connected models of operational systems. System-theoretic safety emphasizes control, feedback, and emergent hazards in complex sociotechnical systems.

The distinctive contribution of constitutional synchronization is the treatment of all the following as one governed correspondence ecology:

- identity;
- semantics;
- transition law;
- realization;
- observation;
- evidence;
- standing;
- authority;
- historical continuity;
- dependency;
- projection;
- configuration-level admissibility.

It also treats external reachability, revocation, public projection, and resynchronization as first-class consequences of correspondence judgments.

---

15. Propositions

The framework advances the following propositions.

Proposition 1: Execution success does not imply constitutional synchronization

$$\text{ExecutionSuccess}(R) \not\Rightarrow \text{Synchronized}(R)$$

A process may complete while running unadmitted bytes, violating semantics, exercising revoked authority, or producing misleading projections.

Proposition 2: Cryptographic identity is necessary but insufficient

$$\text{DigestMatch}(R) \not\Rightarrow \text{MeaningMatch}(R)$$

Nor does it imply authority, standing, correspondence, or admissibility.

Proposition 3: Local synchronization does not imply configuration synchronization

$$(\forall C_i : \text{Synchronized}(C_i)) \not\Rightarrow \text{Synchronized}(\Sigma)$$

Proposition 4: Synchronization is claim- and purpose-relative

A system may be synchronized for bounded experimental use and unsynchronized for production reliance.

Proposition 5: Divergence and insufficiency are distinct

$$\text{NotProvenSynchronized} \not\Rightarrow \text{ProvenDivergent}$$

Proposition 6: Projection is constitutionally consequential

A projection that misstates standing or authority changes the practical behavior of the system even when underlying code is correct.

Proposition 7: Resynchronization is a governed successor transition

A lawful repair preserves predecessor identity, divergence evidence, authorization, and succession.

Proposition 8: Recursive synchronization requires non-self-ratification

A system cannot acquire external standing merely by proposing, witnessing, and admitting itself through one unseparated context.

---

16. Conditions of Inadequacy

The framework should be revised if:

- important forms of system drift cannot be expressed through its correspondence model;
- analysts cannot distinguish divergence from insufficient evidence;
- synchronization states fail to produce meaningfully different remedies;
- configuration-level synchronization adds no explanatory value beyond local correctness;
- projection synchronization proves irrelevant to user or agent behavior;
- constitutional courts merely rename tests without improving jurisdiction or nonclaim clarity;
- synchronization records impose more complexity than the failures they prevent;
- independent analysts cannot reconstruct why a system received a synchronization status;
- the framework cannot handle partial observability or probabilistic correspondence;
- recursive self-hosting necessarily collapses into self-ratification;
- empirical application fails to reveal defects missed by ordinary verification and configuration management.

These conditions make the framework testable rather than merely definitional.

---

17. Research Agenda

17.1 Synchronization corpus

Construct a benchmark of real systems containing:

- stale documentation;
- mismatched binaries;
- semantic interface drift;
- revoked but reachable capabilities;
- outdated tests;
- hidden dependencies;
- stale dashboards;
- locally valid but globally incoherent assemblies.

Measure whether independent analysts can classify the divergence consistently.

17.2 Court discrimination

For every synchronization court, test not only that valid examples pass but that targeted historical and adversarial mutations fail.

17.3 Synchronization-cost measurement

Measure:

- runtime overhead;
- storage overhead;
- context size;
- witness latency;
- false refusal;
- false admission;
- resynchronization cost;
- operator attention.

17.4 Human projection studies

Test whether synchronized projections improve:

- standing reconstruction;
- detection of stale evidence;
- authority understanding;
- appropriate refusal;
- trust calibration.

Also test whether fabricated constitutional structure can counterfeit warrant.

17.5 Self-hosting experiments

Progressively replace hosted constitutional services with admitted native realizations while preserving alternate verification, rollback, and historical replay.

17.6 Configuration courts

Develop graph-based methods for detecting:

- authority amplification;
- semantic contradiction;
- historical incompatibility;
- dependency cycles;
- common-mode witness failure;
- projection inconsistency.

17.7 Synchronization under uncertainty

Extend the framework to probabilistic models, nondeterministic systems, machine-learning realizations, and partially observable environments.

---

18. Conclusion

A software system is not one artifact.

It is an evolving collection of identity-bearing representations through which meaning becomes executable effect. Requirements, models, schemas, code, binaries, runtime behavior, evidence, authority, history, dependencies, and user projections all claim to describe or govern the same system. Their agreement cannot be assumed, and their divergence cannot be reduced to one category of bug.

Constitutional synchronization provides a framework for keeping these representations mutually accountable.

It does not erase the distinction between specification and implementation. It relies upon that distinction. Specification must remain separate enough to constrain realization. Realization must remain separate enough to be witnessed. Observation must remain separate enough to provide evidence. Evidence must remain separate enough from authority that proof does not grant permission. Projection must remain separate enough from constitutional state that its fidelity can be tested.

The breakthrough is therefore not collapse through identity. It is closure through governed correspondence.

A constitutionally synchronized system can answer:

- what it claims to be;

- which exact realization carries that claim;
- how the realization was produced;
- what it actually did;
- which evidence supports reliance;
- which authority permits its use;
- which history made it current;
- which dependencies sustain it;
- what users and agents are being shown;
- and what transition could lawfully restore correspondence when those answers diverge.

Executable specification emerges when those answers are not merely documented but enforced.

The deepest implication is recursive. Once a system can represent, witness, and govern its own synchronization, it can turn its own divergences into bounded engineering work. It can improve implementations without silently amending meaning, revise models without rewriting history, regenerate projections without inventing standing, and replace portions of its host substrate without surrendering constitutional continuity.

A system remains itself not because its code never changes, but because change cannot escape the identities, constraints, evidence, authority, and history that make the successor answerable to the system it claims to continue.

Appendix A – Compact Synchronization Record

```
SynchronizationRecord
    record_id
    subject_identity
    purpose
    context
    disturbance_envelope
    invariant_set

    semantic_model_identity
    transition_model_identity
    realization_identity
    execution_environment_identity
    observation_identity
    evidence_identity
    standing_identity
    authority_identity
    historical_state_identity
    dependency_snapshot_identity
    projection_identity
```

```
    identity_status
    semantic_status
    transition_status
    realization_status
    observation_status
    evidence_status
    authority_status
    temporal_status
    dependency_status
    projection_status
    configuration_status

    overall_disposition
    declared_limits
    residuals
    required_resynchronization
    predecessor_record
    issued_by
    issued_at
    receipt_commitment
```

Appendix B – Court Census Record

```
CourtRecord
    court_identity
    canonical_name
    proposition_established
    explicit_nonclaims
    jurisdiction
    affected_subsystem
    applicable_phase
    certification_band

    current_standing
    synchronization_status
    latest_result
    latest_result_identity
    latest_result_time

    required_evidence
    required_toolchain
    required_environment
    body_access_class
```

```
    dependency_courts

    source_identity
    realization_identity
    receipt_links
    failure_links
    related_courts

    predecessor
    superseded_by
    historical_results
```

Recommended `body_access_class` values:

```
DIGEST_SUFFICIENT
METADATA_SUFFICIENT
SELECTIVE_BODY_ACCESS_REQUIRED
FULL_BODY_ACCESS_REQUIRED
RUNTIME_EXECUTION_REQUIRED
EXTERNAL_WITNESS_REQUIRED
```

Appendix C – Divergence and Response Matrix

Divergence	Default response
Identity divergence	Isolate and fail closed
Semantic divergence	Block composition; reopen model or translation
Specification ahead	Continue candidate realization work
Realization ahead	Quarantine capability; propose model amendment
Witness stale	Renew observation before relying
Evidence stale	Narrow or demote standing
Authority divergence	Remove reachability or renew authority
Temporal misalignment	Reconstruct compatible historical state
Dependency drift	Recompute dependency and configuration courts
Projection stale	Regenerate projection; mark prior view historical
Configuration split-brain	Refuse assembly; identify conflicting subgraphs
Unresolvable under model	Generate ontology or modeling exigence

---

Corpus-Wide Reference Positioning

This paper argues that software systems rarely possess one authoritative representation of themselves. Its nearest external comparison fields are distributed, governance, crypto. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are

cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Convergence and consensus establish agreement under a fault model; they do not establish that the agreed state is true, authorized, desirable, or semantically equivalent. A written constitution can legitimate domination as easily as constrain it unless amendment, participation, enforcement, and appeal remain effective in practice. A valid hash, signature, or inclusion proof establishes a bounded cryptographic relation, not meaning, authority, safety, or jurisdiction by itself. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] Leslie Lamport. "Time, Clocks, and the Ordering of Events in a Distributed System." *Communications of the ACM* 21(7), 1978, pp. 558-565. DOI: 10.1145/359545.359563. https://lamport.azurewebsites.net/pubs/time-clocks.pdf **Relationship:** Foundational predecessor for partial order, causality, and distributed event ordering.

[**E2**] Leslie Lamport, Robert Shostak, and Marshall Pease. "The Byzantine Generals Problem." *ACM Transactions on Programming Languages*

and Systems 4(3), 1982, pp. 382-401. DOI: 10.1145/357172.357176. https://doi.org/10.1145/357172.357176 **Relationship:** Predecessor fault model for agreement under adversarial or inconsistent participants.

[**E3**] Marc Shapiro et al. *A Comprehensive Study of Convergent and Commutative Replicated Data Types.* INRIA Research Report RR-7506, 2011. https://inria.hal.science/inria-00555588 **Relationship:** Predecessor convergence model for replicated state; convergence is compared with, but not equated to, legitimacy or semantic agreement.

[**E4**] W3C. *PROV-O: The PROV Ontology.* W3C Recommendation, 30 April 2013. https://www.w3.org/TR/prov-o/ **Relationship:** Adjacent provenance standard; it models entities, activities, and agents, but does not by itself establish the paper's stronger authority or standing claims.

[**E5**] Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action.* Cambridge University Press, 1990. DOI: 10.1017/CBO9780511807763. https://doi.org/10.1017/CBO9780511807763 **Relationship:** Predecessor institutional account of rule-governed collective action and polycentric governance.

[**E6**] Wil Waluchow and Dimitrios Kyritsis. "Constitutionalism." *Stanford Encyclopedia of Philosophy*, substantive revision 2023. https://plato.stanford.edu/entries/constitutionalism/ **Relationship:** Adjacent legal-philosophical account of limited authority; included to expose differences between political and computational constitutionalism.

[**E7**] Lawrence Lessig. *Code: Version 2.0.* Basic Books, 2006. https://codev2.cc/ **Relationship:** Predecessor argument that software architecture regulates behavior; the paper's contribution may be to make such regulation inspectable and revisable.

Ableman corpus

[**A1**] Adam Ableman. *Mosaic Internet Protocol (MIP).* Ableman Corpus current edition; work identity `PAPER.PUB.20115615`;
source SHA-256
`C56A63A0B84816BF8AF602C7467221B467312339ACE5F77DB754F804ABF833C0`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *The Optional Coordinator.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-727469A6944DA256`;
source SHA-256
`92E8F6743B8DDAFF369D2F40120A384DDE273613B12B27D2FC14610469F5A46E`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Constitutional Collective with Optional Overmind (CCOO).* Ableman Corpus current edition; work identity `AMCS-LOCAL-WORK-ED07EA7D91B87C5E`; source SHA-256 `0346FA1B79A2A44A6034DEDCE4D0FA0E5C5CCD953F4AD29A88CEE1F290DCC210`. **Corpus relationship:** priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Communal Computing and Constitutional Governance.* Ableman Corpus current edition; work identity `AMCS-LOCAL-WORK-A6393299A70371DC`; source SHA-256 `302FC6C269369AC4A4A5D8470E56596B6E8813A82530AD9DA64E9FAEEC4D2461`. **Corpus relationship:** priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Ableman Constitutional Governance.* Ableman Corpus current edition; work identity `AMCS-LOCAL-WORK-79353AF163A4B0F1`; source SHA-256 `FB0D9B2D3D54168FFE70B24308D1B2766D6D663ED24502D475E975939BE97C8B`. **Corpus relationship:** priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Constitutional Theory.* Ableman Corpus current edition; work identity `AMCS-LOCAL-WORK-D2D97C4042FD281F`; source SHA-256 `FACC256EDDE5A7D7EA18133CFC9C060CFB8C3811B0221E1969E62347F2BB69CF`. **Corpus relationship:** priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Governance Engineering.* Ableman Corpus current edition; work identity `PAPER.PUB.18749870`; source SHA-256 `372169F315D192733856B1F4A3F1038BBC73F57CF3ED9ACABAE72B965A69C8D0`. **Corpus relationship:** priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Editorial source notes retained from predecessor

Benveniste, A., Caillaud, B., Nickovic, D., Passerone, R., Raclet, J.-B., Reinkemeier, P., Sangiovanni-Vincentelli, A., Damm, W., Henzinger, T. A., & Larsen, K. G. (2015). *Contracts for Systems Design: Theory.* Research Report, INRIA.

Blair, G., Bencomo, N., & France, R. B. (2009). Models@run.time. *Computer, 42*(10), 22–27.

de Alfaro, L., & Henzinger, T. A. (2001). Interface automata. In *Proceedings*

of the 8th European Software Engineering Conference and 9th ACM SIGSOFT International Symposium on Foundations of Software Engineering, 109–120. doi:10.1145/503209.503226.

Garlan, D., Allen, R., & Ockerbloom, J. (1995). Architectural mismatch, or, why it's hard to build systems out of existing parts. In *Proceedings of the 17th International Conference on Software Engineering*, 179–185.

Hoare, C. A. R. (1969). An axiomatic basis for computer programming. *Communications of the ACM, 12*(10), 576–580, 583. doi:10.1145/363235.363259.

Lamport, L. (1994). The Temporal Logic of Actions. *ACM Transactions on Programming Languages and Systems, 16*(3), 872–923.

Lamport, L. (2003). *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.

Leucker, M., & Schallhart, C. (2009). A brief account of runtime verification. *Journal of Logic and Algebraic Programming, 78*(5), 293–303.

Leveson, N. G. (2012). *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press.

Adam Ableman (2026). *Constraint-Bearing Composability: An Ontology of Constraint Carriage, Component Standing, and Architectural Admission*. Constitutional Computing.

Meyer, B. (1992). Applying "Design by Contract." *Computer, 25*(10), 40–51. doi:10.1109/2.161279.

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM, 15*(12), 1053–1058. doi:10.1145/361598.361623.

Standing and Claim Boundary

This paper is a conceptual, formal, or architectural contribution unless it explicitly binds a statement to a cited external source or an exact carried evidence artifact. Unbound statements about implementation, validation, performance, safety, universality, or proof are research claims or design obligations, not established operational facts. Corpus-internal references establish conceptual lineage and comparison, not independent external confirmation.

References

The entries below are corpus-internal comparison and dependency sources. Their inclusion does not establish external validation of this paper's empirical, safety, novelty, or performance claims.

Adam Ableman *Constraint-Bearing Composability.* Zenodo, 2026. doi:10.5281/zenodo.21632061

Adam Ableman *A Theory of Agentic Observation.* Zenodo, 2026. doi:10.5281/zenodo.20437813

Adam Ableman *CARA: Constrained Artificial Reasoning Architecture.* Zenodo, 2025. doi:10.5281/zenodo.18099715

Adam Ableman *Constitutional Computation.* Zenodo, 2026. doi:10.5281/zenodo.20624488

Adam Ableman *Fractal Governance Architecture.* Zenodo, 2026. doi:10.5281/zenodo.20754189

Adam Ableman *Governed Semantic Compilation: Lawful Continuation, Refusal Semantics, and Epistemic Promotion in Large Language Models.* Zenodo, 2026. doi:10.5281/zenodo.18453681

Adam Ableman *MCIR: A Canonical Intermediate Representation for Governed Transformation.* Zenodo, 2026. doi:10.5281/zenodo.21095303

Adam Ableman *Semantic Coordinate Identity Tokenization (SCIT).* Zenodo, 2026. doi:10.5281/zenodo.20532070

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus

License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.23848756